

GroveLEDBarPWM — Full API Reference

Version 1.9.0

Seeed Grove LED Bar V2.1 / MY9221

Arduino-compatible boards, including ESP32-C3

1. Overview

GroveLEDBarPWM provides PWM control for ten LEDs on the Grove LED Bar V2.1. V1.9 adds global output brightness and process-indicator flashing while retaining the underlying LED brightness and graph level.

V1.9 includes individual LED brightness, graph/level display, global brightness, configurable LED flashing, 100% flash override, Moving Dot, Knight Rider, Pulse/Breathe, and non-blocking operation.

2. Installation

Include the library with:

```
#include <GroveLEDBarPWM.h>
```

Create and initialise the bar:

```
GroveLEDBarPWM bar(2, 3);

void setup()
{
    bar.begin();
}
```

3. Graph Level

setLevel(level)

Sets the graph level from 0 to 10.

```
bar.setLevel(6);
```

clear()

Turns all LEDs off.

```
bar.clear();
```

4. Individual LED Brightness

setBrightness(index, brightness)

Sets a logical LED's PWM brightness from 0 to 255.

```
bar.setBrightness(5, 128);
```

setBrightnessPercent(index, percent)

Sets an individual LED from 0 to 100 percent.

```
bar.setBrightnessPercent(5, 50);
```

getBrightness(index) / getBrightnessPercent(index)

Reads the stored brightness of an individual LED.

```
uint8_t pwm = bar.getBrightness(5);
uint8_t percent = bar.getBrightnessPercent(5);
```

Brightness arrays

The library also supports setting and reading all ten LED brightness values using percentage or PWM arrays.

5. Global Brightness — V1.9

setGlobalBrightness(percent)

Sets the overall output brightness without changing the underlying LED brightness values or graph level. Range: 0–100%.

```
bar.setGlobalBrightness(40);
```

getGlobalBrightness()

Returns the current global brightness percentage.

```
uint8_t brightness = bar.getGlobalBrightness();
```

Global brightness acts as an output multiplier. Returning it to 100% restores normal output.

6. Flashing LED — V1.9

setFlashSpeed(milliseconds)

Sets the configured flash rate.

```
bar.setFlashSpeed(250);
```

flashLED(index)

Starts a non-blocking flash on the selected LED. During the flash-on phase the LED reaches 100% brightness, independent of global brightness. Between flashes it returns to its normal brightness.

```
bar.flashLED(5);
```

isFlashing()

Indicates whether a flash is active.

```
if (bar.isFlashing()) {  
    // Flash is active  
}
```

Flash behaviour

The flash is an overlay on the graph/LED state. Normal LEDs follow their stored brightness and global brightness. The flashing LED temporarily overrides the output and reaches 100%.

7. Graph + Flash Process Indicator

```
bar.setLevel(6);  
bar.setGlobalBrightness(40);  
bar.setFlashSpeed(250);  
bar.flashLED(5);
```

LEDs 0–5 represent the level. Normal output is globally dimmed, while LED 5 flashes to 100% and returns to its normal brightness between flashes.

If the graph level is reduced so the selected flashing LED is outside the active graph, the flash stops. Increasing the level again does not automatically restart it; call `flashLED()` explicitly.

8. Graduated Brightness

setGraduated(true/false)

Enables or disables graduated graph brightness.

setGraduationMin(percent)

Sets the minimum graduation percentage.

9. Green-to-Red Direction

setGreenToRed(true/false)

Controls the logical green-to-red orientation.

10. Smooth Transitions

setTransition(true/false)

Enables or disables smooth transitions.

setTransitionSpeed(value) / setTransitionTime(milliseconds)

Controls transition timing.

isTransitioning()

Returns true while a transition is active.

11. Effects

V1.9 retains the V1.8 effects:

EFFECT_MOVING_DOT

EFFECT_KNIGHT_RIDER

EFFECT_PULSE

```
bar.startEffect(GroveLEDBarPWM::EFFECT_PULSE);  
bar.stopEffect();
```

setEffectSpeed(value)

Sets the effect timing.

isEffectRunning()

Reports whether an effect is active.

12. Pulse/Breathe

setPulseRange(minimumPercent, maximumPercent)

Sets the Pulse/Breathe minimum and maximum brightness.

```
bar.setPulseRange(10, 80);  
bar.setEffectSpeed(20);  
bar.startEffect(GroveLEDBarPWM::EFFECT_PULSE);
```

The effect is non-blocking and breathes between the configured limits.

13. update()

Call update() repeatedly from loop() to service transitions, flashing and effects.

```
void loop()  
{  
    bar.update();  
  
    // Other application code can run here.  
}
```

The library is designed to remain non-blocking.

14. Complete V1.9 Example

```
#include <GroveLEDBarPWM.h>  
  
GroveLEDBarPWM bar(2, 3);  
  
void setup()  
{  
    bar.begin();  
    bar.setLevel(6);  
    bar.setGlobalBrightness(40);  
    bar.setFlashSpeed(250);  
    bar.flashLED(5);  
}
```

```
void loop()
{
    bar.update();
}
```

15. API Summary

Core: begin(), setLevel(), clear(), show()

Brightness: setBrightness(), setBrightnessPercent(), getBrightness(), getBrightnessPercent(), array brightness functions

Graph: setGraduated(), setGraduationMin(), setGreenToRed()

Transitions: setTransition(), setTransitionSpeed(), setTransitionTime(), isTransitioning()

Flashing: setFlashSpeed(), flashLED(), isFlashing()

Global brightness: setGlobalBrightness(), getGlobalBrightness()

Effects: setEffectSpeed(), startEffect(), stopEffect(), isEffectRunning(), update()

Pulse/Breathe: setPulseRange(), getPulseMinimum(), getPulseMaximum()

16. Version History

V1.6 — Moving Dot effect.

V1.7 — Knight Rider effect.

V1.8 — Pulse/Breathe effect with configurable minimum and maximum brightness.

V1.9 — Global brightness and process-indicator flashing integrated with graph and LED brightness.

End of API Reference

GroveLEDBarPWM V1.9.0